Secure SDLC and Coding Standard

Ref: 2.2
Product: iConnect Plus
Organisation: Cache Digitech
Classification: Internal — Confidential
Version: 2.0
Date: 2026-11-03
Deployment model: On-premises (Cache Digitech premises) · VPN for remote access
Audience: Software engineers, reviewers, QA, security champions

---

1. Purpose

Define the **secure software development lifecycle (SDLC)** and **coding standards** for iConnect Plus. Source may live on GitHub; **production runtime** is on Cache Digitech premises.

2. SDLC phases

Phase	Activities	Security gates
Requirements	BRD/FRD in `docs/01_BRD`, `docs/02_FRD`	Privacy/security stories identified
Design	SDD, Architecture Lock compliance	Threat notes for new integrations
Implementation	Feature branches, PR review	Lint, type-check, architecture CI
Test	Unit + integration tests	No secrets in tests
Release	Tagged deploy to on-prem host	Change record (§2.4)
Operate	Monitor, patch, backup	VAPT remediation (§2.6)

3. Branching and review

Rule	Detail
Default branch	`main` on GitHub `origin`
PR required	**Template / Pending sign-off** (org policy)
Reviewers	Module owner + platform for cross-cutting
Pre-commit	Hooks as configured in repo
Architecture CI	Permission catalog, ORM import rules, OpenAPI

4. Coding standards — backend (Python)

Topic	Standard
Style	Ruff / project formatter settings
Types	Type hints on public service methods
SQL	SQLAlchemy 2.0; no raw SQL in routers
Secrets	Never commit; use `settings` from env
Crypto	Use platform encryption helpers; no custom ciphers

Errors	HTTPException with safe messages (no stack to client)
Dependencies	Pin in `requirements`; SBOM §2.7

5. Coding standards — frontend (TypeScript)

Topic	Standard
Stack	Next.js 16+, Tailwind, ShadCN
UI system	`ui-ux-pro-max` + `design-system/enterprise-erp-platform/`
Auth	Cookie-based session; no token in localStorage
Permissions	Mirror API codes in `modules.ts`
XSS	React escaping; sanitize rich text per module policy

6. Forbidden patterns (Architecture Lock)

Forbidden	Alternative
Business logic in routers	Services
Cross-module ORM access	Platform ports / facades
Physical DELETE on business data	Soft delete
Direct UI → database	API only
NestJS / Prisma / Mongo primary	Locked stack
Bypass workflow/audit/notify	Outbox engines

7. Security testing in SDLC

Activity	When	Tool / method
SAST	CI on PR	**Template / Pending sign-off**
Dependency scan	CI / release	**Template / Pending sign-off**
DAST	Pre-release on UAT	**Template / Pending sign-off**
VAPT	Periodic	§2.6 tracker

8. Documentation hierarchy (before code)

1. `docs/01_BRD`
2. `docs/02_FRD`
3. `docs/03_SDD`
4. `docs/04_DBS`
5. `docs/05_ARCHITECTURE_LOCK`

9. Training

Topic	Audience	Status
Secure coding (OWASP Top 10)	All engineers	**Template / Pending sign-off**

On-prem secrets handling	Platform + ops	**Template / Pending sign-off**

10. Related documents

Ref	Document
2.1	Application Security Requirements
2.4	Change and Release Records
2.5	Source Code Access Control

Approval: Template / Pending sign-off